Datamining Project Report

Apolline PICARD & Valentine LE MEUR – DIA 5

GitHub: https://github.com/valentiiiine/Web_datamining_project/tree/main

This report presents the full pipeline of our project on Grand Slam tennis. We started with a small set of web pages, cleaned the text, extracted entities and relations, built an RDF knowledge graph, aligned it with Wikidata, expanded it, tested symbolic and embedding-based reasoning, and finally used the graph in a RAG system that generates SPARQL from natural language.

1. Data Acquisition and Information Extraction (01_data_collection.ipynb)

We chose Grand Slam tennis because it is a domain with clear entities and meaningful relations. Players, tournaments, surfaces, countries, and editions appear often enough to build a graph, but the domain is still rich enough to create ambiguity. This makes it a good test case for a small but complete knowledge engineering pipeline.

The screenshots below show the logic of the first stage: define a corpus, respect web constraints, collect useful pages, and verify that the text is rich enough for later NLP processing.

```
SEED_URLS = [
    "https://en.wikipedia.org/wiki/Grand_Slam_(tennis)",
    "https://en.wikipedia.org/wiki/Australian_Open",
    "https://en.wikipedia.org/wiki/French_Open",
    "https://en.wikipedia.org/wiki/Wimbledon_Championships",
    "https://en.wikipedia.org/wiki/US_Open_(tennis)",
    "https://en.wikipedia.org/wiki/2023_Wimbledon_Championships_%E2%80%93_Men%27s_singles",
    "https://en.wikipedia.org/wiki/2022_French_Open_%E2%80%93_Men%27s_singles",
    "https://en.wikipedia.org/wiki/2021_US_Open_%E2%80%93_Men%27s_singles",
    "https://en.wikipedia.org/wiki/Rafael_Nadal",
    "https://en.wikipedia.org/wiki/Novak_Djokovic",
    "https://en.wikipedia.org/wiki/Carlos_Alcaraz",
]

print(f"Project root: {PROJECT_ROOT}")
print(f"Number of seed URLs: {len(SEED_URLS)}")
```

```
Project root: /Users/vincentlemeur/Documents/S8/DIA/Datamining/Project_datamining_bis
Number of seed URLs: 11
```

Figure 1. Seed URLs used for the corpus.

Figure 1 shows the list of 11 seed URLs used at the beginning of the project. The list combines general pages, such as the Grand Slam overview and the four major tournaments, with more focused pages about specific tournament editions and important players. This was a deliberate choice. If we had only used tournament pages, the graph would have been too event centered. If we had only used player pages, the graph would have lacked context about tournaments and surfaces. By mixing both types of pages, we created a corpus where the same facts can appear from different viewpoints. That redundancy is useful because it increases the chance of extracting stable relations such as who won, where a player is from, or which surface a tournament uses.

```python
robots_url = "https://en.wikipedia.org/robots.txt"
robots_response = requests.get(robots_url, timeout=20)
print(f"robots.txt status: {robots_response.status_code}")
print("First 20 lines of robots.txt:\n")
print("\n".join(robots_response.text.splitlines()[:20]))
```

```
robots.txt status: 403
First 20 lines of robots.txt:

Please set a user-agent and respect our robot policy https://w.wiki/4wJS. See also https://phabricator.wikimedia.org/T400119.
```

Figure 2. robots.txt response from Wikipedia.

Figure 2 is important because it shows that crawling is not only a technical problem. The response tells us that Wikipedia expects a user-agent and asks users to respect its robot policy. Even though this is a small project, this step matters because a data pipeline should be reproducible and responsible. In practice, this screenshot also helped us understand that a naive request can fail or be blocked. So the project had to handle access politely instead of assuming that every page could simply be downloaded without constraints.

```python
MIN_WORDS = 500
records, crawl_stats = crawl_urls(SEED_URLS, min_words=MIN_WORDS)
save_jsonl(records, str(OUTPUT_PATH))

print("Crawling finished.")
print(json.dumps(crawl_stats, indent=2))
print(f"Saved useful pages to: {OUTPUT_PATH}")
```

```
INFO | Fetching https://en.wikipedia.org/wiki/Grand_Slam_(tennis)
INFO | Fetching https://en.wikipedia.org/wiki/Australian_Open
INFO | Fetching https://en.wikipedia.org/wiki/French_Open
INFO | Fetching https://en.wikipedia.org/wiki/Wimbledon_Championships
INFO | Fetching https://en.wikipedia.org/wiki/US_Open_(tennis)
INFO | Fetching https://en.wikipedia.org/wiki/2023_Wimbledon_Championships_%E2%80%93_Men%27s_singles
INFO | Fetching https://en.wikipedia.org/wiki/2022_French_Open_%E2%80%93_Men%27s_singles
INFO | Fetching https://en.wikipedia.org/wiki/2021_US_Open_%E2%80%93_Men%27s_singles
INFO | Fetching https://en.wikipedia.org/wiki/Rafael_Nadal
INFO | Fetching https://en.wikipedia.org/wiki/Novak_Djokovic
INFO | Fetching https://en.wikipedia.org/wiki/Carlos_Alcaraz
Crawling finished.
```

Figure 3. Crawl execution over the selected URLs.

Figure 3 confirms that the crawler actually visited the selected pages and stored the cleaned content. At this stage we are not yet building a graph, we are checking whether the input has enough words (more than 500) and whether the pipeline can process every source without breaking. A complete crawl here means the later stages are built on a coherent corpus.

```
rows = load_jsonl(str(OUTPUT_PATH))
collection_df = pd.DataFrame(rows)
collection_df[["title", "url", "word_count"]].sort_values("word_count", ascending=False).head(11)
```

	title	url	word_count
8	Rafael Nadal	https://en.wikipedia.org/wiki/Rafael_Nadal	30221
9	Novak Djokovic	https://en.wikipedia.org/wiki/Novak_Djokovic	29771
10	Carlos Alcaraz	https://en.wikipedia.org/wiki/Carlos_Alcaraz	17216
3	Wimbledon Championships	https://en.wikipedia.org/wiki/Wimbledon_Champi...	13670
0	Grand Slam (tennis)	https://en.wikipedia.org/wiki/Grand_Slam_(tennis)	9685
4	US Open (tennis)	https://en.wikipedia.org/wiki/US_Open_(tennis)	5656
1	Australian Open	https://en.wikipedia.org/wiki/Australian_Open	5528
2	French Open	https://en.wikipedia.org/wiki/French_Open	5011
6	2022 French Open %E2%80%93 Men%27s singles	https://en.wikipedia.org/wiki/2022_French_Open...	1803
7	2021 US Open %E2%80%93 Men%27s singles	https://en.wikipedia.org/wiki/2021_US_Open_%E2...	1351
5	2023 Wimbledon Championships %E2%80%93 Men%27s...	https://en.wikipedia.org/wiki/2023_Wimbledon_C...	862

Figure 4. Word counts after cleaning and minimum-length filtering.

Figure 4 shows the effect of the cleaning pipeline. The most informative pages are the player biographies, especially Rafael Nadal and Novak Djokovic, which contain very large amounts of text. The general tournament pages also remain rich enough to be useful. By contrast, some edition-specific pages are much shorter. This is exactly why the minimum length threshold matters: not every page is equally valuable for information extraction.

2. Entity extraction and normalization (02_information_extraction.ipynb)

After collecting the cleaned pages, we applied spaCy's transformer pipeline to identify named entities and help with relation extraction. The objective was not to extract everything possible, but to keep entities that make sense for this domain: mainly persons, organizations or tournaments, countries, and dates.

```
nlp = load_spacy_model("en_core_web_trf")
print("Loaded spaCy model successfully.")
print("Pipeline components:", nlp.pipe_names)
```

```
Loaded spaCy model successfully.
Pipeline components: ['transformer', 'tagger', 'parser', 'attribute_ruler', 'lemmatizer', 'ner']
```

Figure 5. spaCy model loading and pipeline components.

Figure 5 shows that we used a transformer-based English model rather than a lighter rule-based pipeline. This choice gave us access to NER and dependency parsing in the same workflow. It is useful because relation extraction depends not only on detecting names, but also on understanding who does what in a sentence. The pipeline components listed here are also a reminder that the extraction stage is layered: tokenization, parsing, and named entity recognition work together, and errors in one layer can affect the next one.

```python
normalization_examples = pd.DataFrame(
    {
        "raw_form": ["Roland Garros", "Roland-Garros", "Wimbledon", "U.S. Open", "Novak Djokovic"],
        "normalized_form": [
            normalize_entity_name("Roland Garros"),
            normalize_entity_name("Roland-Garros"),
            normalize_entity_name("Wimbledon"),
            normalize_entity_name("U.S. Open"),
            normalize_entity_name("Novak Djokovic"),
        ],
    }
)
display(normalization_examples)
print("Generic entity terms filtered when isolated:", sorted(GENERIC_ENTITY_TERMS))
```

	raw_form	normalized_form
0	Roland Garros	French Open
1	Roland-Garros	French Open
2	Wimbledon	Wimbledon Championships
3	U.S. Open	US Open
4	Novak Djokovic	Novak Djokovic

```
Generic entity terms filtered when isolated: ['championship', 'championships', 'court', 'final', 'open', 'seed']
```

Figure 6. Examples of normalization for tennis entities.

Figure 6 is one of the most meaningful screenshots in the first part of the project because it shows why raw extraction is not enough. The same real-world entity may appear under several forms: Roland Garros and Roland-Garros are mapped to French Open, and Wimbledon is normalized to Wimbledon Championships. Without this step, the graph would contain duplicate nodes that refer to the same thing. That would make later querying harder and would also hurt embeddings, because the signal would be split across several similar but disconnected entities. The list of filtered generic terms is also useful. It shows that we actively removed vague words that look relevant in tennis but do not identify a unique entity on their own.

```python
entities_df, triples_df = process_corpus(records, nlp)
entities_df.to_csv(ENTITIES_PATH, index=False)
triples_df.to_csv(TRIPLES_PATH, index=False)

print(f"Saved {len(entities_df)} entities to {ENTITIES_PATH}")
print(f"Saved {len(triples_df)} triples to {TRIPLES_PATH}")
```

```
Saved 15138 entities to /Users/vincentlemeur/Documents/S8/DIA/Datamining/Project_datamining_bis/data/interim/extracted_entities.csv
Saved 761 triples to /Users/vincentlemeur/Documents/S8/DIA/Datamining/Project_datamining_bis/data/interim/extracted_knowledge.csv
```

Figure 7. Number of extracted entities and candidate triples.

Figure 7 gives the first clear measure of scale. The pipeline extracted 15,138 entities and 761 candidate triples. The gap between these two numbers is not surprising. Entity extraction is easier and more frequent than relation extraction, because a page may mention many names and dates without expressing an explicit relation in every sentence. This also tells us that later stages must be selective. A graph built directly from all detected text patterns would be too noisy, so normalization and filtering become central steps rather than optional polishing.

```
entity_type_df = compute_entity_type_distribution(entities_df)
display(entity_type_df)
```

	entity_type	count
0	PERSON	6454
1	DATE	5904
2	ORG	1915
3	GPE	865

Figure 8. Distribution of entity types in the extracted corpus.

Figure 8 helps us understand what kind of corpus we actually built. Persons dominate the extraction, which is expected because tennis pages talk a lot about players and opponents. Dates are also very frequent, reflecting the importance of tournament years, match history, and biography timelines. The lower number of GPE entities suggests that location information exists but is not the main focus of the pages. This distribution is useful because it already predicts the shape of the future graph: it will naturally be centered on players, with time and event context around them.

```
relation_df = relation_distribution(triples_df)
display(relation_df)
```

	relation	count
0	playedAgainst	433
1	won	170
2	hasSurface	108
3	participatedIn	31
4	fromCountry	19

Figure 9. Most frequent extracted relations before RDF construction.

Figure 9 shows the relation distribution before RDF construction. The relation playedAgainst appears much more often than the others, which is coherent with tennis narratives where players are often discussed through rivalries and matchups. Won is the second most common relation, which is valuable because it connects players to tournament success. HasSurface, participatedIn, and fromCountry are less frequent, but they are semantically important because they link different types of entities. This imbalance is a good example of why relation counts must be interpreted carefully: a frequent relation is not always the most important one for answering questions.

At this stage, three ambiguity problems became clear. First, tournaments can be tagged as organizations even though they function as events in our ontology. Second, a year such as 2022 can be a normal date in one sentence but an edition identifier in another. Third, co-occurrence can create weak links between players simply because they appear in the same sentence. These issues are exactly why the next stage cannot just store extracted text as it is. The graph needs modeling choices.

3. Knowledge Base Construction (03_kg_construction.ipynb)

The second major stage of the project was to turn noisy extracted information into a real knowledge graph. This means choosing URIs, deciding classes, defining predicates, and making sure that the same thing is represented only once. In this step the project moves from text processing to knowledge engineering.

```python
uri_examples = pd.DataFrame(
    {
        "label": ["Rafael Nadal", "Wimbledon Championships", "Grass", "Spain"],
        "role": ["Player", "Tournament", "Surface", "Country"],
    }
)
uri_examples["uri"] = uri_examples.apply(lambda row: role_to_uri(row["role"], row["label"]), axis=1)
display(uri_examples)
print("Slug example for 'Carlos Alcaraz':", slugify("Carlos Alcaraz"))
```

	label	role	uri
0	Rafael Nadal	Player	http://example.org/tennis/player/Rafael_Nadal
1	Wimbledon Championships	Tournament	http://example.org/tennis/tournament/Wimbledon...
2	Grass	Surface	http://example.org/tennis/surface/Grass
3	Spain	Country	http://example.org/tennis/country/Spain

```
Slug example for 'Carlos Alcaraz': Carlos_Alcaraz
```

Figure 10. Examples of URI construction for different roles.

Figure 10 illustrates an important design choice: entities receive different URI patterns depending on their role. A player, a tournament, a surface, and a country are not just labels placed in one flat namespace. Separating them at the URI level makes the graph more readable and easier to maintain. It also reflects an ontological decision: these entities are not interchangeable, even if they may appear in similar text contexts. The slug example for Carlos Alcaraz shows the practical side of this design. Stable URI creation is a small detail, but it is essential for reproducibility.

```python
artifacts = build_kg_artifacts(triples_df)
serialize_graph(artifacts.ontology_graph, str(ONTOLOGY_PATH))
serialize_graph(artifacts.instance_graph, str(INITIAL_KG_PATH))
serialize_graph(artifacts.instance_graph, str(INITIAL_NT_PATH), fmt="nt")

with STATS_PATH.open("w", encoding="utf-8") as file:
    json.dump(artifacts.stats, file, indent=2)

print(f"Cleaned triples retained for RDF construction: {len(artifacts.cleaned_triples)}")
print(f"Ontology triples: {len(artifacts.ontology_graph)}")
print(f"Instance graph triples: {len(artifacts.instance_graph)}")
```

```
Cleaned triples retained for RDF construction: 671
Ontology triples: 41
Instance graph triples: 1859
```

Figure 11. RDF construction results after cleaning candidate triples.

Figure 11 shows the transition from extracted triples to RDF artifacts. Out of 761 candidate triples, 671 were retained for graph construction. This difference is healthy rather than problematic. It means the system did not blindly trust all extraction output. Instead, it cleaned and normalized the data before inserting it into the graph. The final instance graph contains 1,859 triples, which is much larger than the retained relation count because the graph also includes typing information and structural facts. This is a good example of how RDF modeling adds semantic structure beyond raw subject-predicate-object extraction.

```python
with STATS_PATH.open("r", encoding="utf-8") as file:
    stats = json.load(file)

print(json.dumps(stats, indent=2))
```


```json
{
  "total_triples": 1859,
  "total_entities": 243,
  "total_relations": 6,
  "predicate_uris": [
    "http://example.org/tennis/ontology/editionYear",
    "http://example.org/tennis/ontology/fromCountry",
    "http://example.org/tennis/ontology/hasSurface",
    "http://example.org/tennis/ontology/participatedIn",
    "http://example.org/tennis/ontology/playedAgainst",
    "http://example.org/tennis/ontology/won"
  ],
  "class_counts": {
    "Player": 187,
    "Tournament": 4,
    "Match": 0,
    "Surface": 3,
    "Country": 13,
    "TournamentEdition": 31
  }
}
```


Figure 12. Initial graph statistics before external expansion.

Figure 12 gives a compact summary of the private graph before alignment and expansion. The graph has 1,859 triples, 243 entities, and 6 relations. At first sight, six relations may look too small, but in fact it reflects a useful modeling decision. A compact predicate set reduces semantic fragmentation. Instead of keeping many near-duplicate relations, we mapped information into a smaller, more stable ontology. The class counts are also meaningful. Players dominate the graph, which follows naturally from the corpus, while TournamentEdition introduces useful temporal granularity. The absence of Match instances suggests that the current graph focuses more on stable entities and outcomes than on full match-by-match event modeling.

A strong point of this stage is that the ontology remains small and understandable. For a student project, this is often better than creating a very large schema that looks impressive but is difficult to justify. The graph already supports useful queries about origins, wins, participation, and surfaces, which are exactly the kinds of questions later tested in the RAG step.

4. Alignment (04_alignment_and_expansion.ipynb)

After building the private graph, we aligned it with Wikidata. The goal was not to replace our graph, but to anchor it in a larger public knowledge base and use that alignment to enrich it with more facts.

```python
alignment_artifacts = build_alignment_and_expansion(triples_df, initial_kg)

alignment_artifacts.alignment_df.to_csv(ALIGNMENT_TABLE_PATH, index=False)
alignment_artifacts.alignment_graph.serialize(ALIGNMENT_TTL_PATH, format="turtle")
alignment_artifacts.expanded_graph.serialize(EXPANDED_TTL_PATH, format="turtle")
alignment_artifacts.expanded_graph.serialize(EXPANDED_NT_PATH, format="nt")
with EXPANDED_STATS_PATH.open("w", encoding="utf-8") as file:
    json.dump(alignment_artifacts.expanded_stats, file, indent=2)

print(f"Saved alignment table to: {ALIGNMENT_TABLE_PATH}")
print(f"Saved alignment graph to: {ALIGNMENT_TTL_PATH}")
print(f"Saved expanded graph to: {EXPANDED_TTL_PATH}")
```

```
/opt/anaconda3/lib/python3.13/site-packages/rdflib/plugins/serializers/nt.py:39: UserWarning: NTSerializer always uses UTF-8 encoding.
  warnings.warn(
Saved alignment table to: /Users/vincentlemeur/Documents/S8/DIA/Datamining/Project_datamining_bis/kg_artifacts/alignment_table.csv
Saved alignment graph to: /Users/vincentlemeur/Documents/S8/DIA/Datamining/Project_datamining_bis/kg_artifacts/alignment.ttl
Saved expanded graph to: /Users/vincentlemeur/Documents/S8/DIA/Datamining/Project_datamining_bis/kg_artifacts/expanded_kg.ttl
```

Figure 13. Saved artifacts produced by the alignment and expansion step.

Figure 13 confirms that alignment and expansion were treated as explicit project artifacts, not as temporary notebook output. Saving the alignment table, alignment graph, and expanded graph separately is good practice because each file answers a different question. The table explains which matches were chosen, the alignment graph stores semantic links, and the expanded graph contains the new knowledge used later for embeddings and RAG.

	private_entity	entity_role	external_uri	candidate_label	candidate_description	confidence	decision
0	New Zealand	Country	http://www.wikidata.org/entity/Q664	New Zealand	island country in the southwest Pacific Ocean	1.00	accept_alignment
1	Jamaica	Country	http://www.wikidata.org/entity/Q766	Jamaica	island state in the Caribbean Sea	1.00	accept_alignment
2	Spain	Country	http://www.wikidata.org/entity/Q29	Spain	manual high-confidence seed	0.99	accept_alignment
3	Australia	Country	http://www.wikidata.org/entity/Q408	Australia	manual high-confidence seed	0.99	accept_alignment
4	David Ferrer	Player	http://www.wikidata.org/entity/Q191476	David Ferrer	Spanish tennis player (born 1982)	1.00	accept_alignment
5	Dominic Thiem	Player	http://www.wikidata.org/entity/Q88762	Dominic Thiem	Austrian male tennis player	1.00	accept_alignment
6	Gaël Monfils	Player	http://www.wikidata.org/entity/Q186429	Gaël Monfils	French tennis player	1.00	accept_alignment
7	Kei Nishikori	Player	http://www.wikidata.org/entity/Q311222	Kei Nishikori	Japanese tennis player	1.00	accept_alignment
8	John Isner	Player	http://www.wikidata.org/entity/Q53566	John Isner	American tennis player	1.00	accept_alignment
9	Casper Ruud	Player	http://www.wikidata.org/entity/Q18810082	Casper Ruud	Norwegian tennis player	1.00	accept_alignment
10	Juan Martín del Potro	Player	http://www.wikidata.org/entity/Q180535	Juan Martín del Potro	Argentine tennis player	1.00	accept_alignment

Figure 14. Sample of the alignment table with external URIs and confidence scores.

Figure 14 is a strong piece of evidence that the linking step was not done blindly. The table shows the private entity, its role, the candidate external URI, a label, a short description, a confidence value, and the final decision. This is important because entity linking is one of the easiest places to introduce serious errors. For example, if a player is linked to the wrong person or if a country name is matched incorrectly, graph expansion can import wrong facts at a large scale. The presence of confidence scores makes the process more transparent. It shows that we do not pretend that every match is equally reliable.

```python
alignment_summary = alignment_df['decision'].value_counts().rename_axis('decision').reset_index(name='count')
display(alignment_summary)
```

	decision	count
0	accept_alignment	42
1	manual_review	14
2	create_local_entity	4

Figure 15. Summary of alignment decisions.

Figure 15 summarizes the decision logic. Forty-two entities were accepted directly, fourteen required manual review, and four were created locally. This balance is actually a good sign. If all entities had been accepted automatically, the process would probably be too optimistic. If too many entities had to be rejected, the linking approach would be too weak. The presence of a manual review group shows a realistic middle ground: some cases are straightforward, while others need human judgment. The local entities are also important because not every useful concept in our project has to come from Wikidata.

```
predicate_text_search_query = """
SELECT ?property ?propertyLabel WHERE {
  ?property a wikibase:Property .
  ?property rdfs:label ?propertyLabel .
  FILTER(CONTAINS(LCASE(?propertyLabel), "surface"))
  FILTER(LANG(?propertyLabel) = "en")
}
LIMIT 20
"""

surface_property_rows = run_select_query(predicate_text_search_query)
surface_property_df = pd.DataFrame(
    [
        {
            'property': row['property']['value'],
            'propertyLabel': row['propertyLabel']['value'],
        }
        for row in surface_property_rows
    ]
)
display(surface_property_df.head(10))
```

	property	propertyLabel
0	http://www.wikidata.org/entity/P3013	surface tension
1	http://www.wikidata.org/entity/P765	surface played on
2	http://www.wikidata.org/entity/P2856	EU Surface Water Body Code
3	http://www.wikidata.org/entity/P7015	surface gravity
4	http://www.wikidata.org/entity/P7083	surface roughness
5	http://www.wikidata.org/entity/P10614	has surface

Figure 16. Example of predicate search in Wikidata for alignment.

Figure 16 shows predicate alignment at work. Instead of guessing an external predicate, we queried Wikidata properties containing the word surface and inspected the candidates. This is more thoughtful than mapping by intuition alone. Property alignment is not just a naming exercise. It decides whether our local relation really means the same thing as a public one, or whether it is only close in meaning. That distinction matters because claiming full equivalence is stronger than saying that one predicate is only related or narrower.

The expansion strategy used in the project is best described as controlled growth. Rather than importing everything connected to an aligned entity, we expanded only through useful predicates. This keeps the graph large enough for embeddings but avoids the common problem of graph bloat, where a project imports many facts that are technically correct but irrelevant to the final questions.

After the expansion, the expanded graph as the next size:


```
{
  "total_triples": 60443,
  "total_entities": 29830,
```

"total_relations": 139
}

5. Reasoning and Knowledge Graph Embeddings (05_reasoning_and_expansion.ipynb)

Once the expanded graph was ready, we used it in two different reasoning settings. The first one is symbolic reasoning with rules. The second one is statistical reasoning with embeddings. These two approaches are often presented as opposites, but in practice they answer different needs.

Rule-based reasoning is precise and explainable because every new fact comes from an explicit logical pattern. Embedding-based reasoning is less transparent, but it can capture regularities that were never written as rules. For that reason, the project benefits from looking at both.

```python
results_df = pd.read_csv(KGE_RESULTS_PATH)
display(results_df)
```

Python

	model	embedding_dim	learning_rate	batch_size	epochs	MRR	Hits@1	Hits@3	Hits@10	dataset_size	train_triples	valid_triples	test_triples
0	TransE	48	0.01	256	20	0.067383	0.027473	0.058941	0.133866	20k	18064	935	1001
1	DistMult	48	0.01	256	20	0.099189	0.051948	0.098402	0.180819	20k	18064	935	1001
2	TransE	48	0.01	256	20	0.066676	0.024364	0.060731	0.142601	50k	44451	2758	2791
3	DistMult	48	0.01	256	20	0.129661	0.065389	0.136689	0.243103	50k	44451	2758	2791
4	TransE	48	0.01	256	20	0.060739	0.019342	0.056871	0.134238	full	52317	3378	3464
5	DistMult	48	0.01	256	20	0.119199	0.059613	0.119226	0.235566	full	52317	3378	3464

Figure 17. Link prediction results for TransE and DistMult across three dataset sizes.

Figure 17 presents the main embedding results. Two models were tested, TransE and DistMult, on three dataset sizes: 20k, 50k, and the full dataset. Several patterns appear immediately. First, DistMult performs better than TransE on all three splits, especially on MRR and Hits@10. Second, performance generally improves when moving from 20k to 50k, which suggests that the models benefit from more graph structure. Third, the full dataset does not produce a dramatic improvement over 50k. This is a very interesting result because it suggests that more data is useful only up to a point. If the extra triples add noise or redundancy, the gain may become small.

The best score in this table is DistMult on the 50k split, with an MRR around 0.13 and Hits@10 around 0.24. These values are moderate, not spectacular, but they are reasonable for a project graph that still contains extraction noise and simplified predicate alignment. In that sense, the results are informative rather than disappointing. They show that the model learns some structure, but they also reflect the limits of the source graph.

Looking at the embedding results, it becomes clear why DistMult performs better than TransE on our dataset. This mainly comes from the types of relations presents in the graph. For example, the relation *playedAgainst* is symmetric: if player A played against player B, then player B also played against player A. DistMult handles this kind of relation well because it models interactions in a symmetric way. On the other hand, TransE represents relations as translations in vector space, which makes it less adapted to symmetric patterns. However, not all relations behave like this. The relation *won* is asymmetric, since if a player wins a tournament, the opposite is not true. This shows a limitation of DistMult, which cannot model asymmetry properly. In our case, since symmetric relations are quite frequent, DistMult ends up performing better overall, but this also highlights that model performance strongly depends on the structure of the knowledge graph.

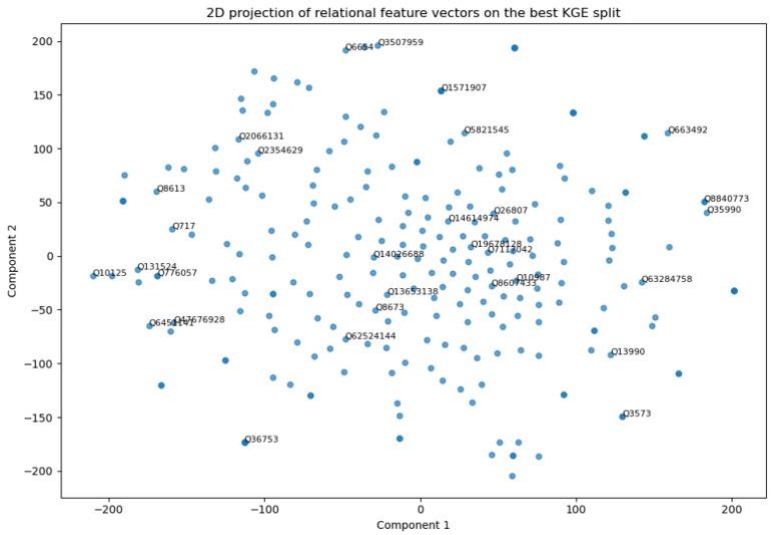

Figure 18. Two-dimensional projection of learned embedding vectors.

Figure 18 offers a visual view of the learned embedding space. The projection does not show very clean clusters, which is itself a useful observation. In a perfectly structured graph, we would expect stronger grouping by type or semantic role. Here, the spread of points suggests that the model learned some local structure but not a very clear global geometry. This is consistent with the nature of our graph. The graph mixes high-quality relations with weaker ones coming from automatic extraction, so the embedding space also reflects that mixed quality. Another reason is that our ontology is relatively simple, with a limited number of relation types, which reduces the semantic signals available for clustering. However, the lack of clear clusters does not mean the embeddings are useless. It is still possible that the model captures local similarities between entities. To better understand this, we can look at nearest neighbors of specific players, such as Rafael Nadal, and check whether the retrieved entities are semantically related. This kind of analysis is more reliable than visual inspection alone and helps confirm whether the embeddings learned meaningful patterns despite the weak clustering.


```
{
  "http://example.org/tennis/player/Rafael_Nadal": [
    [
      "http://example.org/tennis/player/Roger_Federer_172",
      0.9999477915594184
    ],
    [
      "http://example.org/tennis/player/Medvedev",
      0.9998875517706254
    ],
    [
      "http://example.org/tennis/player/Vajda",
      0.9998111006195326
    ],
    [
      "http://example.org/tennis/player/Martin_Klizan",
      0.9997526407850743
    ],
    [
      "http://example.org/tennis/player/Serena",
      0.9996916481010876
    ]
  ],
```


Figure 19. Nearest Neighbors of Rafael Nadal in the Embedding Space

Figure 19 represents the nearest neighbors of Rafael Nadal in the embedding space. This results provide useful insight into what the model has learned. The closest entity is Roger Federer, which is a very coherent result since both players are among the most successful tennis players and share a long rivalry. Other neighbors such as Daniil Medvedev and Martin Kližan are also professional tennis players, which shows that the model captures the general concept of "tennis player." However, some results are less precise. For example, the presence of Serena suggests that the model groups players together based on shared context, even across different categories such as men's and women's tennis. Additionally, the very high similarity scores (all close to 1) indicate that the embedding space may not be well spread, which can reduce the model's ability to clearly distinguish between entities. Overall, this analysis shows that the embeddings capture general semantic similarity, but lack fine-grained structure, likely due to noise and limited relation diversity in the knowledge graph.

6. RAG over RDF and SPARQL (06_rag_chatbot.ipynb)

The final stage of the project consists in using the knowledge graph as a grounded reasoning system through a Retrieval-Augmented Generation (RAG) pipeline. Instead of directly answering questions using a language model, we transform natural language queries into SPARQL queries, execute them on the RDF graph, and return structured results.

This approach is particularly important when using small local LLMs, which tend to hallucinate or produce generic answers. By constraining the model with the schema of the graph and executing queries, the answers become verifiable and grounded in data.

	question	baseline_answer	rag_rows_preview	rows_count	repaired_or_fallback	assessment	comment
0	Which players are from Spain?	Here are some well-known Spanish tennis player...	Carlos Alcaraz | Rafael Nadal	2	True	Correct for current KG	This question is clean and usually works well.
1	Which tournaments did Rafael Nadal win?	A great question about one of the greatest ten...	Australian Open | French Open | Wimbledon Cham...	3	True	Correct for current KG	Grounded answer is correct for the current KG ...
2	Which players participated in the Australian O...	That's a tough one!\n\nUnfortunately, I'm a la...	Jannik Sinner | Novak Djokovic | Rafael Nadal ...	8	True	Correct for current KG	Some extracted player names are noisy, so this...
3	Which players played against Novak Djokovic?	That's a tough one! Novak Djokovic has been pl...	Alexander Zverev | Andre Agassi | Andy Murray ...	20	True	Correct for current KG	The graph contains many valid opponents but al...
4	Which surface is Wimbledon Championships playe...	The answer is Grass. The Wimbledon Championshi...	Clay | Grass | Hard Court	3	True	Correct for current KG	This reveals KG noise because multiple surface...

Figure 20. Evaluation table comparing baseline answers and graph-grounded RAG outputs.

Figure 20 is one of the strongest results in the whole report because it compares baseline answers with grounded RAG outputs on five evaluation questions. The baseline show a complex but relevant answer. The RAG side is much more useful because it returns concrete names from the graph. For example, the system correctly retrieves Carlos Alcaraz and Rafael Nadal for the Spain question, and it returns tournament names for Rafael Nadal's wins.

At the same time, the table also makes the limits of the current graph very visible. The Australian Open participation question returns several good names, but the comment already notes that some extracted players may be noisy. The Wimbledon surface question is even more revealing. The graph returns Clay, Grass, and Hard Court, whereas the expected answer is Grass. This is not a failure of SPARQL generation. It is a failure inherited from graph quality. This example is very useful because it proves that a grounded QA system is only as good as the graph it queries.

```python
evaluation_questions = [
    {
        'question': 'Which players are from Spain?',
        'expected_keywords': ['Carlos Alcaraz', 'Rafael Nadal']
    },
    {
        'question': 'Which tournaments did Rafael Nadal win?',
        'expected_keywords': ['French Open', 'Wimbledon Championships', 'Australian Open']
    },
    {
        'question': 'Which players participated in the Australian Open?',
        'expected_keywords': ['Novak Djokovic', 'Rafael Nadal', 'Roger Federer']
    },
    {
        'question': 'Which players played against Novak Djokovic?',
        'expected_keywords': ['Andy Murray', 'Carlos Alcaraz', 'Alexander Zverev']
    },
    {
        'question': 'Which surface is Wimbledon Championships played on?',
        'expected_keywords': ['Grass']
    },
]
```

Figure 21. Questions.

Figure 21 represents the questions asked to the LLM and the RAG. A key improvement in the RAG pipeline is the implementation of a self-repair mechanism. When a generated SPARQL query fails, the system captures the error message and uses it to prompt the model again. The model receives the schema summary, the original query, and the error, and then produces a corrected version of the query. This corrected query is executed on the knowledge graph.

This mechanism greatly improves the robustness of the system. Even when the initial query is not syntactically correct, the system is often able to recover and return valid results.

	metric	value
0	evaluation_questions	5.0
1	questions_with_grounded_rows	5.0
2	execution_success_rate	1.0
3	repair_or_fallback_rate	1.0
4	rag_graph_triples	1050.0

Figure 22. Global RAG metrics for the evaluation run.

Figure 22 summarizes the global behavior of the RAG evaluation. All five questions produced grounded rows, the execution success rate is 1.0, and the repair or fallback rate is also 1.0. This means that the system managed to execute a valid query for every question, even if some cases needed repair or fallback. From an engineering point of view, this is a strong result. It shows that the system is robust enough to keep working even when the first generated query is not perfect.

The high repair rate is also interesting from an analytical point of view. It means the self-repair mechanism is not just a bonus feature. It is central to making the system usable with a small local LLM. Without repair, the same model would likely fail more often because it might use the wrong prefixes or produce malformed SPARQL.

7. Critical Reflection

The most important lesson from the project is that the pipeline is cumulative. Each step depends on the quality of the previous one. If data acquisition is weak, extraction becomes noisy. If extraction is noisy, RDF construction becomes fragile. If the graph is fragile, embeddings and RAG will expose that weakness very clearly.

One clear example is the surface question in the RAG evaluation. The query mechanism works, but the graph contains multiple surface values for Wimbledon, so the answer is partially wrong. This kind of error is useful because it helps separate problems. The model did not invent the wrong answer. It retrieved what the graph already contained. So the real task for improvement is not only better prompting, but also better graph curation.

The comparison between rule-based and embedding-based reasoning is also meaningful. Rules are precise and easy to justify, but they only infer what we explicitly encode. Embeddings are flexible and can capture patterns across the graph, but they are less interpretable and more sensitive to noise. In this project, neither approach replaces the other. Rules are better when the logic is clear and stable. Embeddings are better when we want to predict missing links or explore similarity.

If the project were extended, the first improvement would be better information extraction. A more domain-specific relation extraction strategy would reduce noisy links and improve both the graph and the downstream tasks. The second improvement would be stricter validation during expansion, especially for predicates that can introduce conflicting facts. The third improvement would be a richer RAG evaluation with more question types, including negative cases and questions that require joins across several parts of the graph.

Overall, the project was successful because it demonstrated an end-to-end workflow and, more importantly, made the dependencies between stages visible. The final system is not perfect, but it is understandable. It answers meaningful questions, shows where noise enters the pipeline, and gives a solid basis for future improvement.

8. Conclusion

This project showed how a small set of web pages can be transformed into a usable knowledge graph and then into a grounded question-answering system. The final value of the project is not only the number of triples or the evaluation scores. It is the understanding that knowledge engineering is a chain of decisions, where modeling, alignment, and cleaning matter as much as the final model.

In the Grand Slam tennis domain, the pipeline already supports useful questions about players, tournaments, surfaces, and participation. The graph is also large enough to support embedding experiments and structured enough to power SPARQL-based RAG. At the same time, the project remains honest about its limits: extraction noise, imperfect alignment, and partial ambiguity are still visible. That honesty is important because it turns the report from a simple demonstration into a real analysis of what works, what fails, and why.

Improvements

SWRL Rule and Reasoning Example

We implemented a simple SWRL rule to demonstrate symbolic reasoning on our knowledge graph. The rule can be expressed as: Player(?p) ∧ fromCountry(?p, Spain) → SpanishPlayer(?p). This rule allows us to infer a new class membership based on existing relations. For example, Rafael Nadal and Carlos Alcaraz are correctly inferred as

Spanish players. This shows that SWRL reasoning is precise and explainable, but also limited to what we explicitly define.

Model Comparison (TransE vs DistMult)

The comparison between TransE and DistMult highlights an important difference in how models capture relations. DistMult consistently performs better on our dataset, especially for metrics like MRR and Hits@10. This is likely because many relations in our graph, such as playedAgainst, are symmetric, which DistMult handles well. In contrast, TransE struggles more with such patterns.

Expansion Strategy Clarification

Our expansion strategy was intentionally controlled. We only expanded from confidently aligned entities and limited the predicates used during SPARQL queries. Without this restriction, the graph would include many irrelevant or noisy facts, which would reduce both interpretability and embedding performance. This shows that scaling a knowledge graph is not just about adding data, but about maintaining semantic relevance.

Baseline vs RAG Comparison

A direct comparison between baseline and RAG clearly shows the benefit of grounding. Without RAG, the model often fails or produces generic answers. With RAG, answers are based on the knowledge graph and therefore more precise, even if sometimes incomplete. This confirms that retrieval is essential for reliable question answering with small language models.

9. Auto evaluation

Overall estimated grade : 15 / 20

Data Acquisition & IE — 2 / 3

- Crawler quality + ethics — 0.5 / 1
The crawler is well implemented with delay, user-agent, filtering, and robots.txt checking. The approach is solid, even if still simple in some edge cases.

- Cleaning + NER — 0.5 / 1
The NER and cleaning pipeline is clearly implemented with domain-specific filters. Some noise still remains in entities and triples.

- Ambiguity & reflection — 1.0 / 1
Ambiguity is well discussed with clear examples and limitations. This part fully meets expectations.

KB Modeling, Alignment & Expansion — 4 / 5

- RDF/ontology design — 1 / 1.5
The ontology is clear and consistent, but remains relatively simple for some domain aspects.

- Entity/predicate alignment — 1 / 1.5
Entity alignment is strong, and predicate alignment is implemented, but still partial and limited in coverage.

- SPARQL expansion strategy — 1.5 / 1.5
The expansion strategy is well designed, controlled, and effective.

- KB statistics — 0.5 / 0.5
Statistics are correctly computed and exported.

Reasoning & KGE — 3 / 4

- SWRL rules + reasoning — 0.5 / 1
Rules are implemented and interpreted correctly, but remain relatively simple.

- ≥ 2 KGE models — 1.0 / 1
Requirement fully satisfied.

- Metrics + analysis — 1.5 / 2
Metrics and comparisons are solid, but analysis could be deeper.

RAG (NL → SPARQL + self-repair) — 3 / 4

- Schema summary + prompting — 0.7 / 1
Schema summary is clear and useful.

- Query execution & self-repair — 0.6 / 1
Execution works and self-repair is implemented, but robustness still depends on question formulation.

- Baseline vs RAG evaluation — 1/ 1
Comparison is present but more demonstrative than fully experimental.

- Demo clarity — 0.7 / 1
The demo is clear and functional after improvements.

GitHub Quality & Reproducibility — 1.5/ 2

- README + instructions — 1 / 1
Clear and well-structured.

- Environment setup — 0.5 / 0.5
Well explained and reproducible.

- Repo organization & commits — 0 / 0.5
Structure is good, but commit history cannot be fully evaluated.We implemted the github at the end.

Final Report — 1.5 / 2

- Clarity, structure, visuals — 1 / 1
Structure is good, but the final report needs stronger visuals and polish.

- Scientific depth & reflection — 0.5 / 1
Good reflection in notebooks, but the report could go deeper.